WORKSHEET **P9** Meeting 9 · DOM, Events, and Interactivity

Mounting Data into the Page and Responding to the User

Web-Based Application Development · SIF302 · Odd Semester 2026/2027

NAME	Laraib Arshad	STUDENT ID	25523275
CLASS	IP INFORMATICS	DATE	15/10/26

Work alone, not in pairs. Every student builds one profile page of their own. You are free to choose the content, but you must be able to explain every line of code you write.

Learning outcome	What is graded today
Students can select page elements, build new elements from data, and respond to user actions with one listener on the parent, so the page changes without a reload.	1. The page runs without errors and no selector returns null. 2. The list of projects comes from the data, not from hand-written HTML. 3. The container is cleared first every time the list is drawn again. 4. One listener on the parent serves every filter button. 5. The form does not reload the page and an error message appears per field.

x

Working rules

Item	The rule
Form of work	Individual. One student, one page, one topic.
Tools	VS Code plus the Chrome or Firefox browser. Chrome DevTools is used throughout from sheet A.
Starting files	Your own work from Meeting 8, meaning profile.html, all the CSS files, and js/app.js, is copied into the folder worksheet-p9/. There is no sample file and no scaffold provided.
Time	90 minutes in class. If you do not finish, continue outside class by 23.59 on the same day.
Submission	The whole contents of worksheet-p9/ are uploaded to your own GitHub repository, inside the semester folder ~/pabw/. Check your repository page, not only the folder on your laptop.

90-minute time budget

Minutes	Sheet	What you work on	If time runs out, stop here
0-18	A	Copy your Meeting 8 result into worksheet-p9/, create js/dom.js, and write down the elements you will fill	Files linked and the element list filled in
18-40	B	Render the project list: map, createElement, textContent, append	The project list appears on the page from the data, not from HTML
40-62	C	The category filter buttons with one listener on their parent	A filter click changes the list content
62-85	D	Your Meeting 3 form gets submit + preventDefault, per-field validation, and error messages	The page does not reload; the error message appears and disappears at the right time
85-110	E	Three hard cases: null from a selector, duplicate listeners, an empty list	Three cases handled + screenshots
110-150	F	Check one by one, one line to remember, an exit ticket, a self-assessment, and the README section	Sheet F filled in and saved

Always work in order. Every sheet assumes the previous one is finished. If you skip ahead, the symptom will be hard to trace because its cause sits on a sheet you have not done. The right-hand column is not a deadline, only a pace marker.

Sheet A. Selecting elements on the page

PICK THE ELEMENT FIRST, THEN TOUCH THE CONTENT

Until Meeting 8 your data was arranged, but it was only visible in the Console. Today that data goes into the page. The first step is not writing but searching: make sure the element you will fill really exists and really is the one you mean.

A.1 Reading the page as a tree and finding elements

Open your page, right-click one area, then choose Inspect. The Elements panel shows the page as a nested tree. What you see there is the shape JavaScript holds, not the HTML file you wrote. Before writing a selector, make sure the id or class name appears in that panel.

What it does	The result	When to use it
querySelector("#list")	The first matching element, or null when there is none	Filling one particular place on the page
querySelectorAll(".filter")	Every matching element, as a NodeList	Handling many buttons or many cards
el.closest("button")	The nearest matching ancestor, starting from the element itself	Finding the button from the element that was really clicked
Array.from(result)	A NodeList turned into an ordinary array	Before using map or filter on a search result

TEST THE SELECTOR IN THE CONSOLE BEFORE WRITING IT INTO A FILE

```
document.querySelector("#list");     // an element, or null
document.querySelectorAll(".filter");  // a NodeList

// NodeList is not an array: map and filter are missing
Array.from(document.querySelectorAll(".filter")).map((t) => t.textContent);
```

A.2 The elements I will fill

Fill in this table before writing code. The id or class names you write here must match the spelling in profile.html letter for letter: that is where the most common null error comes from.

Page area	The selector I use	Filled with	Variable name
Project list	#list	Dynamic room card elements (`<li>`)	container
Filter button row	#filter	Category filter buttons	filterRow
The empty-list message	#empty-msg	Warning text when no rooms match	emptyMsg
The form and each of its fields	#contact-form	Input fields (`#name`, email, etc.)	form

A.3 Creating dom.js and linking it

Create a new file named js/dom.js, then add one more <script> line before </body>. The app.js file from Meeting 8 stays unchanged: it still holds the data and the functions. The dom.js file uses that data to fill the page.

```
  <script type="module" src="js/app.js"></script>
  <script type="module" src="js/dom.js"></script>
</body>
```

File layout	What it is for	What goes wrong when it is wrong
js/dom.js kept separate from app.js	app.js holds the data and the pure functions; dom.js touches the page. This boundary is split again in Meeting 11	Everything written in one file: hard to search when the page does not change
The order of the two <script> lines	app.js first, because dom.js uses data from it	dom.js runs without its data and stops at once
type="module" on both	Both files share the same scope and can use import	Variable names leak into the browser window; import cannot be used

Done	Mark
The folder worksheet-p9/ holds profile.html, all the CSS, and js/app.js from Meeting 8	✔
The file js/dom.js exists and is linked before </body>	✔
Every selector in table A.2 has been tested in the Console and none of them is null	✔
The page is opened through a local server; the Console shows no 404 message	✔
The Elements panel has been used once to confirm the id and class names	✔

When your selector returns null, do not guess. Type the selector in the Console, then compare the result with the Elements panel. Nine out of ten null errors end at one different letter.

SAVE TO GIT. AFTER SHEET A

```
git add .
git commit -m "your message"
git push

# suggested commit message:
# P9 - pick the elements first
```

Sheet A finished? Save the result now instead of waiting for class to end. This save needs no extra step; it only makes sure your work does not depend on one file on one computer.

A - save your changes

How to check: open the Console and call each element variable. None of them is null.

Sheet B. Building elements from data

ONE ITEM OF DATA, ONE SHAPE ON THE PAGE

Now for the real shape of today's work: the project list that has only appeared as a Console table must appear on the page. You are not the one writing the cards; JavaScript is, one card per array item, as many as the data holds.

B.1 createElement, textContent, then append

There are two ways to fill a page, and the choice is not a matter of taste. Pasting HTML text with innerHTML is shorter, but the browser reads it as HTML, including anything a user has typed. Building elements one by one takes a few more lines, and your page cannot be injected.

What it does	Its role	What goes wrong when they are swapped
createElement("li")	Creating a new element that is not on the page yet	A created element does not appear until it is put into a container
textContent = value	Filling text; whatever it holds is treated as text	Using innerHTML: text from the user is run as HTML (XSS)
append(child)	Inserting an element as the last child of the container	append(" ") inserts text, not an element
className = "card"	Giving a class so it appears like the other elements	Without a class, the element exists but has no formatting
DocumentFragment	A temporary container outside the page; the page is drawn once	Without it, the page is drawn again every time one card is added

ONE ARRAY ITEM, ONE CARD

```
  const container = document.querySelector("#list");

function buildCard(project) {
  const li = document.createElement("li");
  li.className = "card";
  li.textContent = project.title;      // text, not HTML
  return li;
}

  projects.forEach((project) => container.append(buildCard(project)));
```

B.2 Clear the container before you fill it again

When a filter button is pressed twice and the list grows every time, there is one cause: the container is never cleared. That clearing line belongs in the first line of the render function, before anything is added.

Line	What to do	Note
container.textContent = ""	Removing all the content of the container with its children	The shortest form; enough for today's work
container.replaceChildren()	Removing every child without touching anything else	Chosen when the container also holds elements that must survive
container.innerHTML = ""	It works, but it forces the browser to parse the HTML again	Not used in this meeting

B.3 Compare your result

Open the Console and compare three things before moving to sheet C. You can check all three yourself without asking.

What is checked	The correct result	The result I got
The number of cards on the page	Equal to the length of projects	4 cards rendered (matches `hotel.roomTypes.length = 4`)
The top card	Its title matches the first data item	"Single Room - Rp 300.000" (matches the first item in `hotel.roomTypes`)
The text inside the card	It appears as text, not as tags that were parsed	Appears as plain text using `textContent` without raw HTML tags

When the list looks right but the Console shows a red message, your work is not finished. An error that remains means some part stopped before the end, and that part will show up on sheet C.

SAVE TO GIT. AFTER SHEET B

```
git add .
git commit -m "your message"
git push

# suggested commit message:
# P9 - list rendered from data
```

Sheet B finished? Save the result now instead of waiting for class to end. This save needs no extra step; it only makes sure your work does not depend on one file on one computer.

B - save your changes

How to check: reload the page without touching the Console. The project list appears at once, and no card is duplicated.

Sheet C. One listener for every button

ONE LISTENER ON THE PARENT, EVERY CHILD SERVED

The first click you must handle is the filter button. The easiest approach for a beginner, attaching a listener to every button, has one weakness: cards and buttons created later have no listener. That is why the listener goes on their parent, once.

C.1 addEventListener and event delegation

A listener on the parent receives every click inside it, including clicks from children that did not exist when the listener was attached. All you have to do is check which element was really clicked.

```
  const filterRow = document.querySelector("#filter");

  filterRow.addEventListener("click", (event) => {
  const button = event.target.closest("button");
  if (!button) return;              // click outside the button, ignore

  const category = button.dataset.category;
  const selected = projects.filter(
   (project) => category === "all" || project.category === category
  );

  render(selected);
});
```

Term	Meaning	Note
event.target	The element that was really clicked	It may be a child of the button rather than the button itself
event.currentTarget	The element the listener is attached to	Inside an arrow function its value stays the parent
closest("button")	Raising the view to the nearest button	When the click is not on a button, the result is null
dataset.category	Reading the data-category attribute	A way to label a button without writing eight if statements

C.2 The active button

The user needs to know which filter is active. Mark the active button with one class rather than with a colour written straight into JavaScript. The styling stays in CSS; JavaScript only adds and removes the class.

```
function markActiveButton(activeButton) {
  document.querySelectorAll("#filter button").forEach((button) => {
    button.classList.toggle("active", button === activeButton);
  });
}
```

State	What must happen	The result I got
A freshly opened page	Every project appears, and the "all" button is marked active	All 4 room cards rendered, and the "All" button has the active highlight class
Click one category	Only projects in that category appear	Only matching rooms (e.g., Single Room) display when the button is clicked
Click an empty category	The container is empty and its message appears, not a blank page	Card container clears and `#empty-msg` displays "No rooms found in this category"

State	What must happen	The result I got
Two quick clicks	The number of cards does not multiply	Cards render cleanly without duplicating because `container.textContent = ""` clears the container first

When the cards multiply every time a button is pressed, do not add new code. First check whether the line that clears the container still sits in your render function.

SAVE TO GIT. AFTER SHEET C

```
git add .
git commit -m "your message"
git push

# suggested commit message:
# P9 - filter works with one listener
```

Sheet C finished? Save the result now instead of waiting for class to end. This save needs no extra step; it only makes sure your work does not depend on one file on one computer.

C - save your changes

How to check: press three filter buttons in turn. The list changes, the number of cards does not multiply, and the Console stays clean.

Sheet D. The render pattern and form validation

THE VIEW IS A FUNCTION OF THE DATA

Until sheet C your view grew from several places: one function added a card, one part hid a message, another line marked the active button. On this sheet all of it is gathered into one render(data) function. After that there is no second place to update.

D.1 One render function per part of the page

A render function has three fixed jobs, and their order cannot be swapped: clear the container, check whether the data is empty, then fill it again. The measure of success is simple: every data change is followed by exactly one render call.

```
function render(list) {
  container.textContent = "";              // 1. clear it first

  if (list.length === 0) {            // 2. check the empty state
    empty.hidden = false;
    return;
  }
  empty.hidden = true;

  list.forEach((project) => container.append(buildCard(project)));   // 3. fill it again
}
```

Rule	The reason	What happens when it is broken
One render function per part of the page	One place must be right; easy to check	The view differs from the data because some part was not updated
The container is cleared on the first line	Old content and new content must not stack	A card multiplies every time render is called
The empty state is handled inside render	The user must never see a silent page	A filter that produces nothing looks like a fault
The listener is attached outside render	Render can be called many times without adding listeners	One click runs many times, and the Console prints repeatedly

D.2 Form validation that answers before submission

Your form has existed on the page since Meeting 3. Now it gets three things: preventDefault so the page does not reload, validation that runs while the user types, and an error message per field. The message names the fix rather than blaming the user.

What it does	Its role	When it runs
event.preventDefault()	Stopping the browser's default submission	On the first line of the submit handler
input.value.trim()	Reading a field value without the spaces at its edges	Every time a field that may be empty is checked
setAttribute("aria-invalid", "true")	Marking a failing field for screen readers	When that field is declared invalid
button.disabled = !valid	Holding submission until every field is valid	Every time the check result changes
field.focus()	Moving the view and the screen reader to the field that needs fixing	After the check finishes and a field is failing

D.3 Check your own result

What is checked	The correct result	The result I got
Submit an empty form	The page does not reload; an error message appears	Page does not reload due to `event.preventDefault()`, and invalid attribute (`aria-invalid="true"`) is set
Correct one field	The message disappears as soon as the content is valid	`aria-invalid="false"` is set, and error styling clears when valid text is provided
The value is only spaces	Still declared invalid	Trimmed value (`.trim() === ""`) detects whitespace and keeps the field invalid
The submit button	Waiting until every field is valid	Button prevents submission until required fields pass validation checks

The form is the only part of the page that can erase all your work in one click. Make sure the page does not reload before you add any validation.

SAVE TO GIT. AFTER SHEET D

```
git add .
git commit -m "your message"
git push

# suggested commit message:
# P9 - render and validation
```

Sheet D finished? Save the result now instead of waiting for class to end. This save needs no extra step; it only makes sure your work does not depend on one file on one computer.

D - save your changes

How to check: submit an empty form, then correct one field. The page does not reload and the error message disappears as soon as the field is valid.

Sheet E. Read the symptom instead of guessing

WHAT FAILS MOST IS NOT THE CODE, BUT THE ORDER

This meeting has one typical kind of failure: a page that appears not to react although the Console is clean. There is no red message to read, only a silent page. This sheet trains you to read that kind of symptom.

E.1 Checking listeners with DevTools

A listener attached twice raises no error. All you see is one action running many times. The Event Listeners panel shows which listener is attached to which element.

What to do	Where	What you see
Right-click, then Inspect	The Elements panel	That element is highlighted in the page tree
The Event Listeners tab	The panel beside Elements	The list of events attached to that element
monitorEvents(el)	Console	Every event on that element is printed as it happens
Count the Console lines	Console, while the button is pressed	One line when it is right; more than one when the listener is duplicated

E.2 A silent page: the four most common causes

The checks below are ordered from the most common cause. Work through them in order; do not jump to the most complicated one.

Symptom	The most common cause	What to do
The page looks normal, and nothing changes	The script runs before its element exists, so the selector returns null	Check the <script> order: app.js then dom.js, both before </body>
The list appears, but a filter changes nothing	The listener is attached to a button that does not exist yet, or to the wrong element	Move the listener to the parent, then check event.target.closest("button")
One click makes the list multiply	addEventListener is called inside the render function	Move the listener outside the render function and attach it once
Empty list with no explanation	The filter produces nothing, and the empty state is not handled	Show the message inside render when list.length === 0
Console: Cannot read properties of null	querySelector does not find the element	Compare the spelling of the id or class with the one in the Elements panel

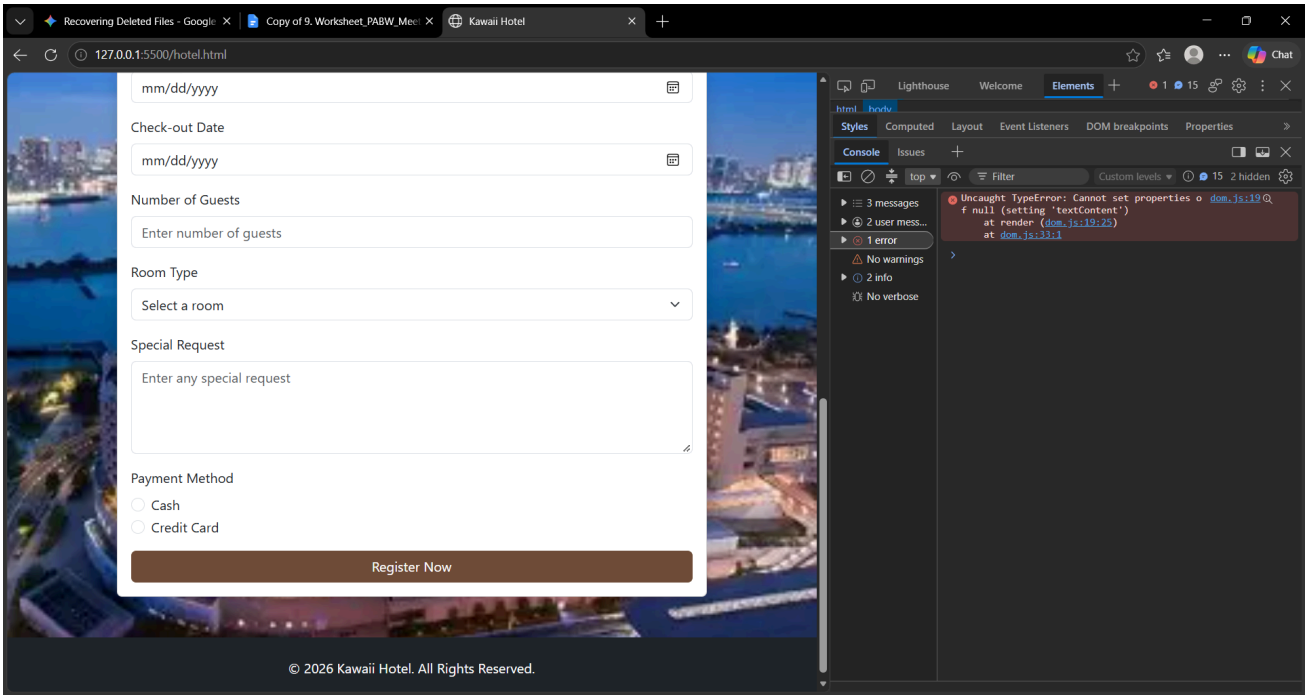
E.3 Three hard cases and how to handle them

What appears	Meaning	What to do
The selector returns null	The id or class name differs, or the element does not exist yet when the script runs	Test the selector in the Console first; check the <script> order
Duplicate listeners after a re-render	The listener is attached again every time the container is filled	Attach the listener once on the parent, outside the render function
The list is empty	The filter produces nothing, or the data has not been read yet	Show the empty-state message; check the array length with console.log

E.4 Record the cases I met

The symptom I saw	The cause	The line I changed
Uncaught TypeError: Cannot set properties of null (setting 'textContent')	document.querySelector("#list") returned null because the container element with id="list" was missing in profile.html.	element with id="list" was missing in profile.html. Added <ul id="list" class="card-list"> into profile.html and verified the selector in dom.js line 4.
Room cards multiplied every time a filter button was clicked	The `render()` function in `dom.js` was missing the step to clear existing list items before appending new ones.	Added `container.textContent = "";` at line 18 in `js/dom.js` to clear previous cards before rendering
Page reloaded / refreshed immediately when clicking the submit button	The form submit event listener was missing the default browser event prevention method.	Added `event.preventDefault();` at line 50 inside the form's submit listener in `js/dom.js`.

Keep three screenshots: one of the silent page with its Event Listeners panel, one after the fix, and one while the filter works. You will need all three on sheet F.



```
}

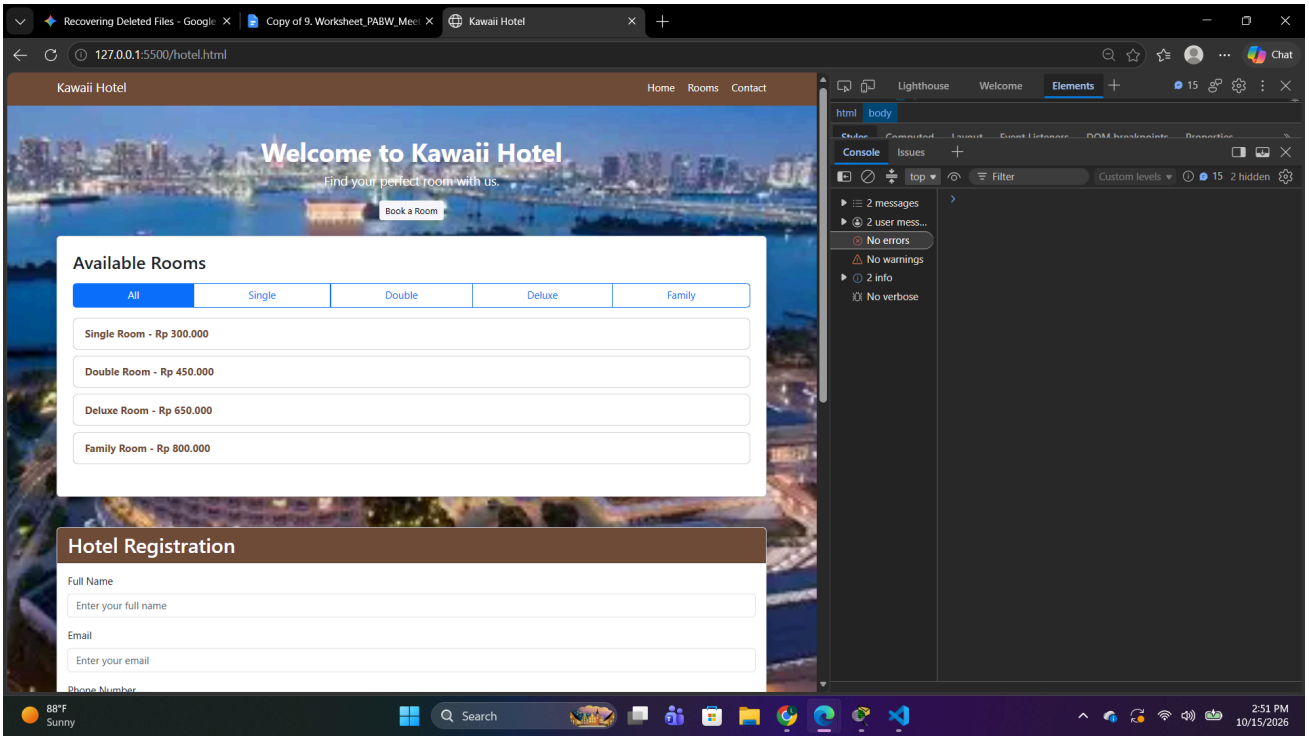
// 5. Form Submit Handling (Sheet D)
if (form) {
    form.addEventListener("submit", (event) => {
    event.preventDefault(); // Prevent page reload

    const nameInput = document.querySelector("#name");
    if (nameInput && nameInput.value.trim() === "") {
        nameInput.setAttribute("aria-invalid", "true");
        alert("Please fill out your name.");
    } else {
        alert("Room booking submitted!");
```

```
}

// 3. Render Function (Sheet B & D)
export function render(list) {
  container.textContent = ""; // Clear existing content to prevent dupl

    if (list.length === 0) {
    emptyMsg.hidden = false;
    return;
    }
```



`

SAVE TO GIT. AFTER SHEET E

```
git add .
git commit -m "your message"
git push

# suggested commit message:
# P9 - read the symptom, do not guess
```

Sheet E finished? Save the result now instead of waiting for class to end. This save needs no extra step; it only makes sure your work does not depend on one file on one computer.

E - save your changes

How to check: the page reacts to every click, and table E.4 is filled with the real cause rather than with "already fixed".

Sheet F. Exit ticket and self-assessment

EXIT TICKET

Work alone, without discussing. This sheet measures your understanding, not how fast you copy a friend's answer.

1. Name one element on your page, the selector you used to get it, and the variable that holds the result.

`Room List Container ()` | Selector: `#list` | Variable: `container`

2. What is the difference between querySelector and querySelectorAll? Why can you not call map on the result of querySelectorAll?

`querySelector` returns only the first matching DOM element, while `querySelectorAll` returns a static `NodeList` of all matching elements. You cannot call `.map()` directly on a `NodeList` because it is an array-like object, not a true JavaScript `Array` (you must convert it using `Array.from()` first).

3. You attached the listener to the parent instead of each button. What is the benefit, and which part of your code proves it?

Benefit: It uses **event delegation**, which improves memory efficiency by managing a single event listener on the parent #filter instead of adding individual listeners to every single button.

Code Proof: filterRow.addEventListener("click", (event) => { const button = event.target.closest("button"); ... }); in js/dom.js

4. What happens if you delete the line that clears the container from the render function? Show it, then write the result down.

Every time `render()` is called when clicking a filter button, new room cards append to the bottom without deleting the existing ones, causing the room cards to multiply/duplicate infinitely on the page.

5. Why must content from a user never be inserted with innerHTML? Write the approach you use instead.

Using `innerHTML` with user input creates a severe **Cross-Site Scripting (XSS)** vulnerability because raw text is parsed as executable HTML/JavaScript. Instead, use **`textContent`** or **`createElement()`**, which safely treats user input strictly as plain text

F.1 Check one by one

Do not hand in your work before running this list. Everything below you can check yourself in five minutes, and all of it is really graded.

✔ Files profile.html, all the CSS, js/app.js, and js/dom.js are in the repository and open with no error in the Console

✔ Selectors every selector returns an element; none of them is null
✔ Rendering from data the project list is built from an array with createElement and textContent, not from hand-written HTML
✔ Container cleared on the first line of the render function, so cards never multiply
✔ Event delegation one listener on the parent serves every filter button; event.target and closest are used correctly
✔ Empty state the page shows a message when the filter produces no content
✔ Form validation the page does not reload, an error message appears per field, and the submit button waits until the content is valid
✔ AI disclosure the README section states which parts were helped by AI and which you did yourself
✔ Git there is more than one commit today, and the commit messages explain their content

F.2 One line to remember

Write one sentence, not for the lecturer but for yourself two weeks from now, about what made you go wrong most often today.

Always ensure HTML element IDs match selector strings in JavaScript exactly, and always clear the parent container before appending new list elements

F.3 Self-assessment

Fill in your estimated mark honestly before handing in this sheet. An honest self-assessment tells you which part is not solid yet. In the Evidence column write the file name or code line the lecturer can open.

☐ AI disclosure the README states which parts were helped by AI and which I did myself
☐ Originality the topic and content of this page are my own, not copied from another student's work

Section	Weight	My mark	Evidence
Selecting and filling elements	20	20	`js/dom.js` (lines 4–7) & `profile.html`
Rendering from data	25	25	`js/dom.js` (`buildCard()` & `render()`, lines 10–27)
Events and event delegation	25	25	`js/dom.js` (`filterRow.addEventListener`, lines 33–47)
Form validation	20	20	`js/dom.js` (`form.addEventListener`, lines 50–61)
Code hygiene, AI disclosure, and evidence	10	10	`README.md` & Sheet E screenshots attached
TOTAL	100	100	

F.4 Notes for the lecturer

The hardest part today:

Debugging `null` selector errors caused by mismatched HTML IDs and understanding event delegation with `event.target.closest("button")`

What I want to discuss next class:

IF someone check this before next clas i wanna say that we should practice a class coz with theoretical things no satisfaction

Signature:  Date: _15/10/26 Finish time: 3:43 pm

SAVE TO GIT. AFTER SHEET F

```
git add .
git commit -m "your message"
git push

# suggested commit message:
# P9 - check, then save
```

Sheet F finished? Save the result now instead of waiting for class to end. This save needs no extra step; it only makes sure your work does not depend on one file on one computer.

F - save your changes

How to check: open the README in your repository. The Meeting 9 section holds the AI disclosure, and the last commit is the newest one.

Appendix A. Marking rubric

MARKING RUBRIC total 100

Aspect	Weight	Mark 100. Very good	Mark 65. Adequate	Mark 35. Poor
Selecting and filling elements	20%	Correct selectors and no null; page content inserted with textContent; classes used for formatting	Some selectors still need fixing, or some content is inserted with innerHTML	No element is selected correctly, or the page does not change at all
Rendering from data	25%	The list is built from an array; the container is cleared first; the card order matches the data order	The list appears, but the cards multiply or their order differs from the data	The list is still hand-written in HTML, or render returns undefined
Events and event delegation	25%	One listener on the parent serves every button; target and closest are used as intended; there are no duplicate listeners	The listener works but is attached per button, or a listener is added on every render	The click changes nothing, or one click runs several actions
Form validation	20%	The page does not reload; a per-field error message names the fix; the button waits until the data is valid	Validation runs, but its messages are vague or the submit button is not held back	The form still reloads the page, or validation is missing
Code hygiene, AI disclosure, and evidence	10%	Consistent naming, a clear boundary between app.js and dom.js; the AI disclosure filled in the README; screenshots attached	The code is tidy but the AI disclosure is incomplete, or the evidence is incomplete	No AI disclosure, or the work is not saved in the repository

What is graded is your own work. Topics differ between students, so marking cannot compare one piece of work with another. That is why what is checked is whether each part works as intended, not whether it matches the sample.

Appendix B. Quick reference and what to do when stuck

QUICK REFERENCE FOR THE DOM memorise these

What it does	Its role	When to use it
querySelector	Taking the first matching element	Finding a single unique element (like `#list` or `#contact-form`) using CSS selectors
querySelectorAll	Taking all matching elements as a NodeList	Selecting multiple matching elements on the page (like `document.querySelectorAll("#filter button")`).
Array.from(NodeList)	Turning a NodeList into an array	Converting a static `NodeList` returned by `querySelectorAll` into a true Array to use higher-order functions like `.map()` or `.filter()`

What it does	Its role	When to use it
createElement	Creating a new element	Dynamically generating new HTML DOM nodes (like `document.createElement("li")`) in memory before inserting them into the page
textContent	Filling text safely	Inserting pure plain text into an element safely without rendering raw text as HTML tags, preventing XSS vulnerabilities.
append	Inserting an element as the last child	Adding a newly created element or text node into a parent container element
replaceChildren()	Emptying the container of every child	Clearing out all child nodes inside an element in a single step (a modern alternative to `container.textContent = ""`).
classList.toggle	Adding or removing a class	Toggling a CSS class on/off based on a boolean condition (e.g., highlighting active filter buttons)
addEventListener	Attaching a listener	Registering an event handler (like `"click"` or `"submit"`) to respond dynamically to user interactions
event.target.closest	Finding the nearest matching ancestor element	Locating the specific button or parent element clicked during event delegation, even if the user clicked an inner child node.
preventDefault()	Stopping the browser's default behaviour	Preventing standard browser actions, such as preventing form submissions from refreshing the entire page
dataset	Reading a data-* attribute	Reading custom HTML `data-*` attributes (like `button.dataset.category`) to inspect metadata attached to DOM elements

WHEN STUCK look for the symptom here

Symptom	The most common cause	What to do
console: Cannot read properties of null	querySelector does not find the element	Compare the spelling of the id or class with the Elements panel
The page does not change when a button is pressed	The listener is attached to the wrong element, or to one that does not exist yet	Attach the listener on the parent and check event.target
The first click works, the second click doubles it	addEventListener is called inside the render function	Move the listener attachment outside render
Cards keep growing in the list	The container is not cleared before it is filled again	Add container.textContent = "" on the first line of render
Empty list with no note	The filter produces nothing	Show the message when list.length === 0
The card content appears as tags instead of text	The content is inserted with innerHTML	Replace it with textContent
The form reloads the page	preventDefault is not called in the submit handler	Call event.preventDefault() on the first line
Validation passes on a field holding spaces	The value is checked without trim()	Test with field.value.trim() !== ""

Symptom	The most common cause	What to do
Console: 404 (Not Found) for dom.js	The path in the src attribute does not match the file's location	Make sure dom.js sits inside the js/ folder, with the same letter case
A change to the copy also changes the original data	An array copied with = instead of [...]	Change it to const copy = [...projects]

The page you build today is the material for the Block 2 CPMK exam later: selecting elements, rendering from data, and event delegation are all tested there. Keep this work tidy in your repository, because you will use it again in Meeting 11.

APPENDIX. SAVING YOUR WORK TO GIT once only, not every meeting

This appendix is not an introduction to Git. The Git and GitHub guide you have used since Meeting 3 still applies in full; this section only repeats the three commands you will use most, because from today there is a new file that must be saved with it.

The three commands you will use all the time

Command	Meaning	When it runs
git add .	Marking every change in this folder for saving	Every time one sheet is finished
git commit -m "message"	Saving that change as one piece of history	Right after git add
git push	Sending that history to GitHub	After the commit, so the lecturer can see it

The dot in git add . That dot means "every file in this folder". Without the dot, Git does not know what to save. The same applies to the dom.js file you just created: it is saved because it sits inside the semester folder.

How often should you save?

There is no right number, but here is the rule of thumb: when you have finished one meaningful thing you can explain, save it. In this meeting that means at least twice: after the list appears from the data, and after the filter and validation are done.

If this happens	What you do
You have written a lot but never committed	Save right now
A wrong commit that is not pushed yet	Fix it and commit again. Do not delete the repository
You have pushed, then found a mistake	Fix the file and commit again; the old history does not disappear and that is normal

What is graded is not the number of commits. Saving often keeps your work from depending on one file on one computer, and in the exam meeting the repository is what decides whether your work can be marked at all.